Laporan Tugas Besar 1

IF3170 Inteligensi Artifisial

Optimasi Pengisian Paket Truk Peti Kemas dengan *Local Search*

Semester I Tahun 2026/2027



Kelompok G07 LaCafeLapo

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

Stevanus Agustaf Wongso	:	13524020
Jonathan Kris Wicaksono	:	13524023
Philipp Hamara	:	13524101

Daftar Isi

1 Deskripsi Persoalan **2**
1.1 Entitas dan Atribut . . . 2
1.2 Fungsi Objektif dan Batasan . . . 2
1.3 Ruang Gerak Pencarian . . . 2

2 Pembahasan **2**
2.1 Representasi State, Evaluator, dan Heuristik . . . 2
2.2 Hill-Climbing . . . 3
2.3 Simulated Annealing . . . 4
2.4 Genetic Algorithm . . . 4
2.5 Metodologi dan Hasil Eksperimen . . . 5
2.6 Visualisasi Hasil . . . 5
2.7 Analisis . . . 8

3 Kesimpulan dan Saran **9**

4 Pembagian Tugas **10**

Referensi **11**

1 Deskripsi Persoalan

Permasalahan yang diselesaikan adalah memilih dan menata paket di dalam satu truk peti kemas agar jumlah *value* paket yang berhasil dimuat maksimum. Setiap paket berbentuk balok, dapat diputar pada tiga sumbu, dan boleh tidak dimuat sama sekali. Truk pada eksperimen utama berukuran $10 \times 20 \times 5$ dengan kapasitas berat 500 unit.

1.1 Entitas dan Atribut

Entitas	**Atribut**	**Makna dan contoh**
Paket	`id`	Identitas unik, misalnya `PKG-01`.
Paket	dimensi (w, l, h)	Lebar, panjang, tinggi dalam unit bilangan bulat, contoh $(3, 4, 2)$.
Paket	orientasi	Permutasi tiga sisi paket dengan enam kemungkinan orientasi.
Paket	*value*, *weight*	Nilai dan berat paket, contoh 85 dan 40 unit.
Paket	`is_fragile`, ETA	Penanda barang pecah-belah dan perkiraan sisa hari kedatangan.
Truk	dimensi (W, L, H)	Ukuran ruang muat, eksperimen memakai $(10, 20, 5)$.
Truk	`max_capacity`	Kapasitas berat maksimum, eksperimen memakai 500 unit berat.

Tabel 1: Entitas dan atribut pada persoalan optimasi muatan.

Koordinat menggunakan bilangan bulat. Orientasi menyatakan sisi paket yang sejajar dengan sumbu x, y, dan z setelah rotasi 90 derajat. Atribut ETA dibaca dari dataset, tetapi tidak masuk ke fungsi objektif maupun batasan pada spesifikasi ini.

1.2 Fungsi Objektif dan Batasan

Untuk state s yang valid, fungsi objektifnya adalah jumlah *value* paket yang berada di dalam truk:

$$f(s) = \sum_{i=1}^{n} v_i \, \mathbf{1}[i \text{ berada di dalam truk}]. \tag{1}$$

State valid harus memenuhi batasan berikut:

1. Paket yang dimuat seluruhnya berada di dalam batas ruang truk, sedangkan paket yang tidak dimuat seluruhnya berada di luar, bukan menembus dinding.
2. Tidak ada dua paket yang menempati sel ruang yang sama.
3. Paket tidak melayang: sedikitnya satu titik pada alasnya mendapat dukungan dari lantai truk ($z = 0$) atau permukaan atas paket lain. Ketentuan ini mengikuti definisi pada spesifikasi, sehingga tidak mensyaratkan seluruh bidang alas tertopang.
4. Paket *fragile* tidak boleh menjadi penyangga bagi paket lain.
5. Jumlah berat paket di dalam truk tidak melebihi kapasitas maksimum.

1.3 Ruang Gerak Pencarian

Initial state dibangkitkan secara acak. Ketiga operator tetangga yang digunakan adalah: (1) menukar penempatan dua paket, (2) memindahkan satu paket ke koordinat lain atau mengubah statusnya antara di dalam dan di luar, dan (3) memutar satu paket 90 derajat terhadap salah satu sumbu. Sebuah state mencatat seluruh paket, posisi dan orientasi paket yang dimuat, status paket yang tidak dimuat, kondisi truk, serta nilai objektifnya.

2 Pembahasan

2.1 Representasi State, Evaluator, dan Heuristik

Representasi program berada di `src/core/models.py`. Objek `Package` menyimpan atribut paket, `Truck` menyimpan dimensi dan kapasitas, lalu `PlacedPackage` menyimpan paket, koordinat sudut kiri-bawah-belakang, orientasi, dan penanda `inside`. Sementara itu, `State` menyimpan truk dan daftar seluruh

paket. Orientasi direpresentasi sebagai permutasi indeks dimensi paket, sehingga enam orientasi yang mungkin dapat dinyatakan tanpa mengubah dimensi asal.

Pemeriksaan batasan dilakukan oleh `Evaluator` di `src/core/evaluator.py`. Program membangun *occupancy grid* berukuran (W, L, H). Nilai 0 berarti sel kosong dan nilai positif menandai paket yang menempatinya. Evaluator memeriksa batas peti, tabrakan, dukungan alas, paket *fragile*, dan kapasitas berat. Nilai state tidak valid adalah -1, sedangkan nilai state valid adalah jumlah *value* paket di dalam truk. Array grid memakai NumPy.

Sesuai definisi pada spesifikasi, sebuah paket tidak dianggap melayang apabila sedikitnya satu sel pada alasnya mendapat dukungan. Implementasi `check_support` menerapkan definisi tersebut:

Listing 1: Pemeriksaan dukungan alas pada `Evaluator`.

```
def check_support(self, grid: np.ndarray, pp: PlacedPackage) -> bool:
    dx, dy, dz = pp.dims()
    x, y, z = pp.position
    if z == 0:
        return True
    return bool(np.any(grid[x : x + dx, y : y + dy, z - 1] != 0))
```

Inisialisasi dan penempatan ulang menggunakan heuristik sederhana pada `src/core/initializer.py`: paket diacak urutannya, peluang mencoba memuat paket ditentukan oleh `fill_ratio`, orientasi dan koordinat (x, y) dipilih acak, lalu program mencari koordinat z terendah yang valid. Kapasitas berat diperiksa sebelum paket benar-benar dimasukkan. Operator tetangga di `src/core/neighbors.py` adalah `swap`, `move`, dan `rotate`. `random_neighbor` memilih salah satunya secara acak. Jika tidak ditemukan penempatan valid, operator dapat menghasilkan state yang tidak berubah.

2.2 Hill-Climbing

Keempat varian Hill-Climbing diimplementasikan, meskipun spesifikasi wajib meminta sedikitnya satu varian. Karena keempatnya mencakup seluruh varian Hill-Climbing pada materi kuliah (*steepest ascent*, *stochastic*, *sideways move*, dan *random restart*), implementasi ini mengambil bonus implementasi seluruh algoritma Hill-Climbing. *Steepest ascent* pada program ini bukan enumerasi seluruh tetangga: setiap iterasi mengambil sampel 30 tetangga acak dan memilih nilai terbaik dari sampel tersebut. *Stochastic* menguji satu tetangga acak per iterasi dan hanya menerima perpindahan yang meningkatkan nilai. *Sideways move* mengizinkan perpindahan dengan nilai sama sampai 20 kali berturut-turut. *Random restart* mengulang Hill-Climbing dari state acak baru.

Varian	**Parameter eksperimen dan kondisi berhenti**
Steepest ascent	30 tetangga per iterasi. Berhenti saat sampel tidak memuat tetangga yang lebih baik atau setelah 1.000 iterasi.
Stochastic	Satu tetangga per iterasi. Berhenti setelah 100 iterasi berturut-turut tanpa perbaikan atau setelah 1.000 iterasi.
Sideways move	30 tetangga per iterasi. Maksimal 20 perpindahan bernilai sama secara berturut-turut dengan batas 1.000 iterasi.
Random restart	30 tetangga per iterasi dengan batas 500 iterasi per pencarian. Parameter `max_restart`=5.

Tabel 2: Parameter Hill-Climbing pada eksperimen.

Berikut potongan loop steepest ascent yang digunakan pada `src/algorithms/hill_climbing.py`:

Listing 2: Pemilihan tetangga terbaik dari sampel pada steepest ascent.

```
it = 0
while it < max_iter:
    it += 1
    best, best_value = _best_neighbor(
        current, evaluator, rng, n_neighbors
    )
    if best_value is not None and best_value > current_value:
        current, current_value = best, best_value
        history.append(current_value)
    else:
        history.append(current_value)
        break
```

Ada detail semantik pada penghitung *random restart* yang perlu diperhatikan saat membaca hasil. Loop dengan `max_restart`=5 menjalankan lima pencarian Hill-Climbing total: pencarian pertama menggunakan state awal yang diberikan, lalu empat pencarian berikutnya menggunakan state acak baru.

Dengan demikian, angka 5 pada keluaran program adalah jumlah *start*/siklus, bukan lima *restart* tambahan setelah state awal. Jumlah iterasi tiap siklus dicantumkan pada Tabel 3. Algoritma Hill-Climbing terutama mengacu pada materi kuliah IF3170 [1, 2, 4].

2.3 Simulated Annealing

Simulated Annealing (SA) memakai jadwal pendinginan geometrik $T_{k+1} = 0{,}995T_k$, dengan $T_0 = 100$, batas bawah 10^{-3}, dan batas maksimum 100.000 iterasi. Untuk perubahan nilai $\Delta E = f(s_{\text{tetangga}}) - f(s_{\text{sekarang}})$, state yang lebih baik selalu diterima. State dengan nilai sama atau lebih buruk diterima dengan peluang $\exp(\Delta E/T)$. Pencarian berhenti ketika suhu turun di bawah batas.

Listing 3: Aturan penerimaan kandidat pada SA.

```
delta = cand_value - current_value
if delta > 0:
    prob = 1.0
    accept = True
else:
    prob = math.exp(delta / t)
    accept = rng.random() < prob
```

Variabel `stuck_count` bertambah setiap kali terjadi 500 iterasi berturut-turut tanpa peningkatan nilai state `current`. Pencacah kemudian direset. Riwayat peluang penerimaan yang diplot adalah peluang penerimaan aktual: nilainya 1 untuk langkah yang membaik maupun langkah bernilai sama, dan $\exp(\Delta E/T)$ untuk langkah yang tidak membaik. Penjelasan SA mengikuti materi kuliah IF3170 [1, 5].

2.4 Genetic Algorithm

Satu individu pada Genetic Algorithm (GA) adalah satu penempatan lengkap semua paket. Populasi awal dibangkitkan dengan `random_state`. Nilai *fitness* menggunakan evaluator yang sama dengan algoritma lain. Evaluator memberi nilai -1 kepada state tidak valid. Pada seleksi *roulette wheel*, hanya fitness positif yang memperoleh bobot. fitness tidak positif tidak dipilih melalui bobot, dan jika seluruh bobot nol induk dipilih secara seragam. Nilai rata-rata yang disimpan untuk plot tetap menggunakan skor evaluator, termasuk -1 untuk individu tidak valid.

Operator *crossover* memotong urutan paket pada satu titik: awalan diambil dari orang tua pertama dan paket lain diurutkan menurut orang tua kedua. Cara ini menjaga agar setiap ID paket muncul tepat sekali, tetapi gabungan posisi dari kedua orang tua masih dapat menghasilkan tabrakan atau state tidak valid. Mutasi menerapkan satu operator tetangga acak. Elitisme membawa satu individu terbaik ke generasi berikutnya. Riwayat GA mencatat fitness maksimum dan rata-rata populasi per generasi.

Listing 4: Seleksi, crossover, mutasi, dan elitisme pada loop GA.

```
for _ in range(generations):
    fitnesses = evaluate_population(population, evaluator)
    top = 0
    for i in range(1, len(population)):
        if fitnesses[i] > fitnesses[top]:
            top = i

    rata = sum(fitnesses) / len(fitnesses)
    result.best_history.append(fitnesses[top])
    result.avg_history.append(rata)
    elite = copy.deepcopy(population[top])
    anak_baru = [elite]
    while len(anak_baru) < population_size:
        bapak = select(population, fitnesses, rng)
        ibu = select(population, fitnesses, rng)
        anak = crossover(bapak, ibu, rng)
        anak_baru.append(mutate(anak, rng))
    population = anak_baru
```

GA memiliki dua parameter eksperimen: ukuran populasi dan jumlah generasi. Desain uji menggunakan tiga ukuran populasi pada 30 generasi, lalu tiga jumlah generasi pada populasi 30. GA dan representasi pencariannya mengacu pada materi kuliah IF3170 [1, 6]. Konsep state dan tetangga mengacu pada [1, 3].

2.5 Metodologi dan Hasil Eksperimen

Dataset `data/fixed_packages.json` berisi 20 paket dengan jumlah *value* 1.550 dan jumlah berat 649 unit. Dimensi truk adalah 10×20×5 dan kapasitasnya 500 unit berat. Inisialisasi memakai rasio pengisian 0,7 (parameter `fill_ratio`). Untuk Hill-Climbing dan SA digunakan seed 0, 1, dan 2. Seed yang sama menghasilkan state awal yang sama untuk algoritma-algoritma tersebut. GA memulai populasi acak tersendiri, sehingga nilai awal GA bukan state awal tunggal yang sama dengan HC/SA.

Setiap varian HC dijalankan tiga kali dan SA dijalankan tiga kali. Untuk GA, uji pertama menahan generasi pada 30 sambil memvariasikan populasi 10, 30, dan 50. Uji kedua menahan populasi pada 30 sambil memvariasikan generasi 10, 30, dan 50. Setiap titik uji memakai seed 0, 1, dan 2. Kombinasi populasi 30/generasi 30 menjadi titik kontrol pada kedua uji, sehingga runner mencatat kombinasi itu dua kali (enam baris pada `summary.csv`, tiga seed per uji). Tabel GA di bawah memisahkan kedua sweep dan tidak menghilangkan pengulangan tersebut.

Durasi adalah waktu pencarian yang diukur dengan `time.perf_counter`. waktu visualisasi dan penulisan berkas tidak termasuk. Seluruh hasil run yang tersimpan pada `output/experiments/summary.csv` berstatus valid. File JSON per run memuat nilai awal/akhir, riwayat, serta pemetaan paket awal dan akhir. File PNG memuat visualisasi state awal, state akhir, dan plot riwayat. Plot SA juga memiliki plot peluang penerimaan. File lengkap terdapat di `output/experiments/`. Visualisasi dibuat dengan Matplotlib.

Algoritma (seed)	**Awal → akhir**	**Iterasi**	**Durasi (s)**	**Metrik tambahan**
HC steepest (0)	1050 → 1050	1	0,0058	berhenti tanpa tetangga sampel yang lebih baik
HC steepest (1)	625 → 1160	7	0,0207	–
HC steepest (2)	900 → 1195	4	0,0123	–
HC stochastic (0)	1050 → 1200	209	0,0223	100 iterasi tanpa perbaikan berturut-turut di akhir
HC stochastic (1)	625 → 1165	219	0,0226	100 iterasi tanpa perbaikan berturut-turut di akhir
HC stochastic (2)	900 → 1210	165	0,0194	100 iterasi tanpa perbaikan berturut-turut di akhir
HC sideways (0)	1050 → 1235	32	0,1158	28 langkah sideways total
HC sideways (1)	625 → 1195	36	0,1156	28 langkah sideways total
HC sideways (2)	900 → 1200	26	0,0856	22 langkah sideways total
HC random restart (0)	1050 → 1210	16	0,0537	5 siklus. Iterasi/siklus: [3, 4, 1, 6, 2]
HC random restart (1)	625 → 1185	18	0,0516	5 siklus. Iterasi/siklus: [8, 2, 3, 1, 4]
HC random restart (2)	900 → 1215	30	0,0932	5 siklus. Iterasi/siklus: [7, 7, 3, 6, 7]
SA (0)	1050 → 1215	2297	0,2876	stuck=3
SA (1)	625 → 1220	2297	0,2911	stuck=2
SA (2)	900 → 1225	2297	0,2796	stuck=3

Tabel 3: Hasil per run untuk seluruh varian Hill-Climbing dan SA.

Sweep	**Seed 0: awal → akhir, durasi**	**Seed 1: awal → akhir, durasi**	**Seed 2: awal → akhir, durasi**
Pop 10, gen 30	1200 → 1215, 0,0441 s	1155 → 1180, 0,0395 s	1195 → 1195, 0,0420 s
Pop 30, gen 30 (kontrol sweep populasi)	1205 → 1210, 0,1141 s	1190 → 1225, 0,1139 s	1195 → 1230, 0,1151 s
Pop 50, gen 30	1205 → 1210, 0,1843 s	1190 → 1210, 0,1881 s	1210 → 1210, 0,1676 s
Pop 30, gen 10	1205 → 1210, 0,0526 s	1190 → 1225, 0,0452 s	1195 → 1215, 0,0418 s
Pop 30, gen 30 (kontrol sweep generasi)	1205 → 1210, 0,1176 s	1190 → 1225, 0,1155 s	1195 → 1230, 0,1148 s
Pop 30, gen 50	1205 → 1210, 0,1905 s	1190 → 1225, 0,2592 s	1195 → 1240, 0,2986 s

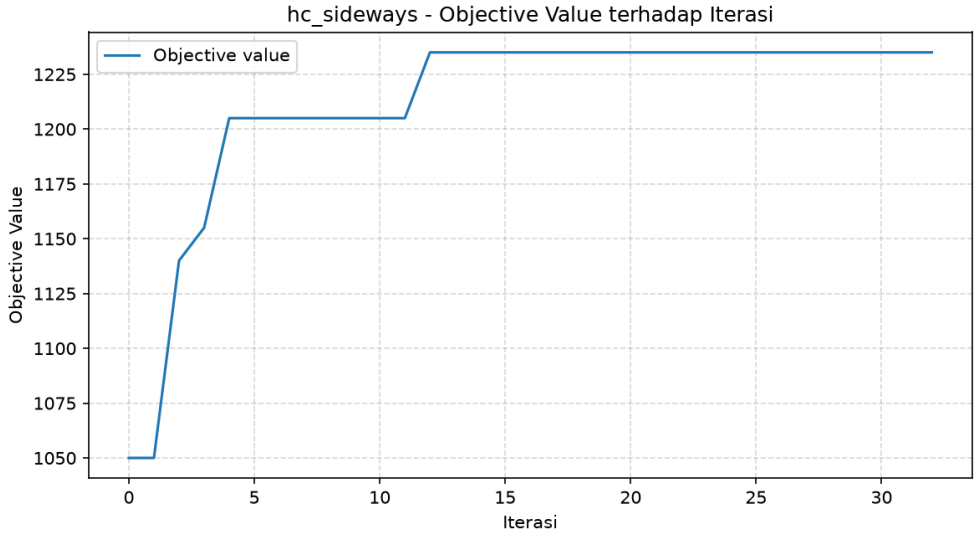
Tabel 4: Hasil GA per seed. Setiap sel memuat nilai terbaik populasi awal, nilai akhir, dan durasi run.

2.6 Visualisasi Hasil

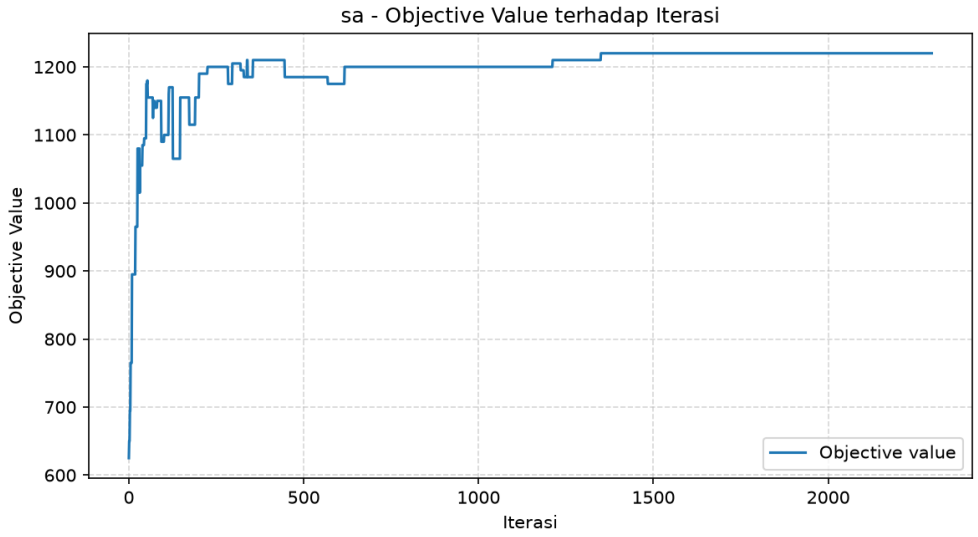
Gambar 1–5 menampilkan contoh plot dan state. Plot dan visualisasi untuk setiap seed lain disimpan bersama JSON hasilnya di `output/experiments/`. Dengan demikian state awal dan akhir tidak hanya tersaji sebagai angka ringkasan, tetapi juga sebagai pemetaan dan gambar yang dapat diperiksa.

Algoritma/konfigurasi	Best	Rentang final	Rata-rata final	Rata-rata durasi (s)
HC steepest	1195	1050–1195	1135,0	0,0129
HC stochastic	1210	1165–1210	1191,7	0,0215
HC sideways	1235	1195–1235	1210,0	0,1057
HC random restart	1215	1185–1215	1203,3	0,0662
SA	1225	1215–1225	1220,0	0,2861
GA pop 10/gen 30	1215	1180–1215	1196,7	0,0419
GA pop 30/gen 10	1225	1210–1225	1216,7	0,0465
GA pop 30/gen 30	1230	1210–1230	1221,7	0,1152
GA pop 30/gen 50	1240	1210–1240	1225,0	0,2494
GA pop 50/gen 30	1210	1210–1210	1210,0	0,1800

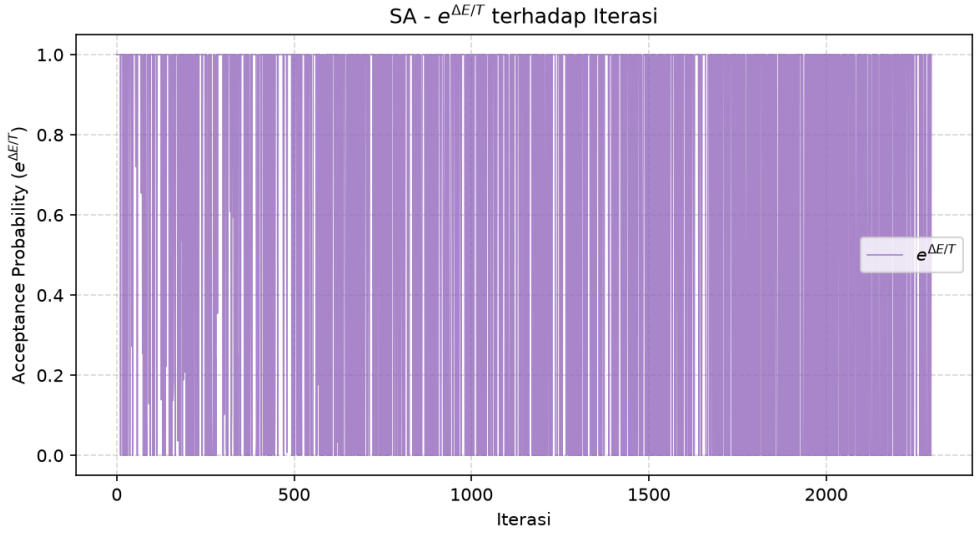
Tabel 5: Ringkasan nilai akhir dan durasi. Baris GA pop 30/gen 30 menggabungkan enam run yang tercatat pada dua sweep. Konfigurasi lain masing-masing memiliki tiga run.



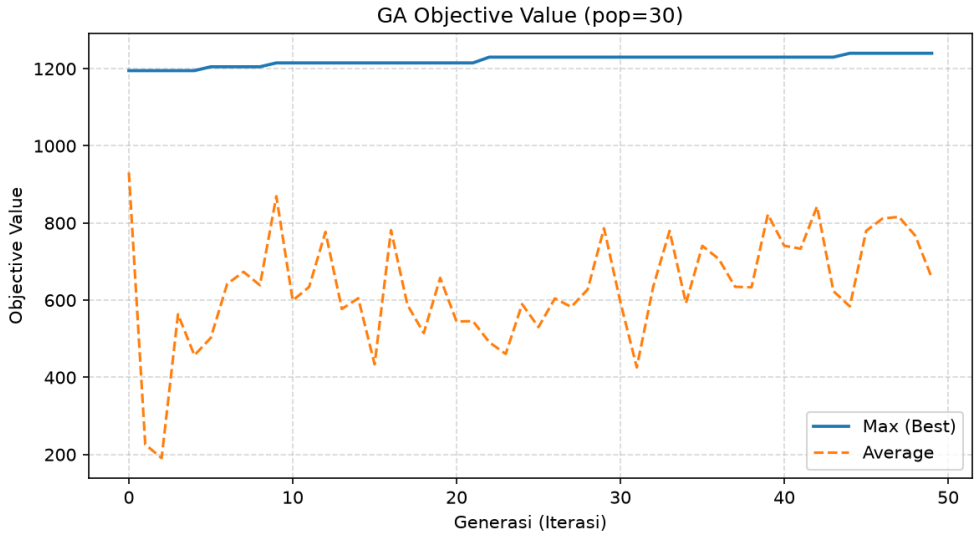
Gambar 1: Riwayat nilai objektif HC sideways, seed 0. Perpindahan sideways dapat menambah iterasi tanpa menaikkan nilai objektif.



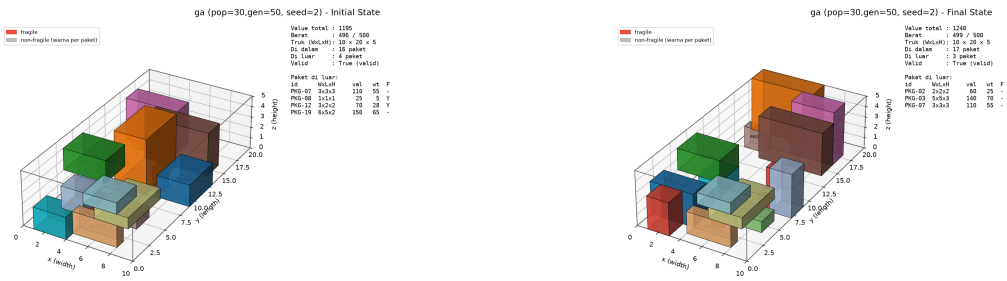
Gambar 2: Riwayat nilai objektif SA, seed 1. Kurva menunjukkan nilai state saat ini, bukan hanya nilai terbaik sepanjang pencarian.



Gambar 3: Peluang penerimaan kandidat SA, seed 1. Nilai 1 juga muncul untuk langkah membaik atau bernilai sama, sehingga plot tidak harus turun monoton.



Gambar 4: Nilai maksimum dan rata-rata populasi GA (populasi 30, generasi 50, seed 2). Elitisme menjaga nilai maksimum agar tidak turun antargenerasi.



State awal terbaik State akhir terbaik

Gambar 5: Perbandingan state awal terbaik dan state akhir GA (populasi 30, generasi 50, seed 2). Gambar memperlihatkan konfigurasi truk, paket yang dimuat, dan daftar paket di luar truk.

2.7 Analisis

1. **Nilai terbaik yang ditemukan.** Nilai tertinggi pada eksperimen adalah 1240, dicapai GA dengan populasi 30, 50 generasi, seed 2. Jumlah *value* seluruh paket adalah 1550, sehingga angka tersebut hanya dapat dibandingkan dengan batas atas kasar 1550. Eksperimen ini tidak menghitung atau membuktikan optimum global. Karena itu hasil 1240 tidak boleh disebut sebagai nilai yang sudah dekat dengan optimum global tanpa pembanding yang terbukti.

2. **Perbandingan hasil pencarian.** Rata-rata akhir tertinggi diperoleh GA pop 30/gen 50 (1225,0), diikuti SA (1220,0). Nilai terbaik tunggal juga berasal dari GA pop 30/gen 50 (1240). HC sideways memberikan hasil HC terbaik (1235). Steepest ascent seed 0 berhenti pada nilai awal 1050 setelah satu iterasi, konsisten dengan kemungkinan bahwa sampel 30 tetangganya tidak memuat state yang lebih baik. SA memeriksa satu tetangga per iterasi dan dapat menerima langkah yang tidak membaik, sedangkan GA mencari dalam populasi dan mempertahankan individu terbaik melalui elitisme.

3. **Durasi.** Di antara konfigurasi yang diuji, HC steepest paling singkat (rata-rata 0,0129 s), sedangkan SA paling lama (0,2861 s). Durasi GA meningkat dari rata-rata 0,0419 s pada pop 10/gen 30 menjadi 0,2494 s pada pop 30/gen 50. Ini adalah waktu dinding pada lingkungan pengujian yang sama, bukan perbandingan biaya per iterasi: satu iterasi steepest mengevaluasi 30 tetangga, sedangkan satu iterasi stochastic mengevaluasi satu tetangga dan satu generasi GA mengevaluasi satu populasi.

4. **Konsistensi.** Rentang hasil SA relatif kecil dan berada pada nilai tinggi, yaitu 1215–1225 (rentang 10). Namun, rentang terkecil diperoleh GA pop 50/gen 30: ketiga seed berakhir pada 1210 (rentang 0), yaitu plateau yang konsisten tetapi lebih rendah. Rentang HC steepest adalah 1050–1195, HC stochastic 1165–1210, HC sideways 1195–1235, dan HC random restart 1185–1215. Rentang GA konfigurasi lain adalah 15–35. Perbandingan konsistensi ini bersifat indikatif karena hanya tiga seed digunakan.

5. **Pengaruh ukuran populasi.** Pada 30 generasi, rata-rata final untuk populasi 10, 30, dan 50 berturut-turut adalah 1196,7, 1221,7, dan 1210,0. Populasi 30 mengungguli dua ukuran lain pada dataset ini. Populasi 50 memerlukan waktu lebih lama daripada populasi 30 tetapi tidak menghasilkan nilai akhir yang lebih tinggi pada tiga seed yang diuji.

6. **Pengaruh banyak generasi.** Pada populasi 30, rata-rata final untuk 10, 30, dan 50 generasi berturut-turut adalah 1216,7, 1221,7, dan 1225,0. Kenaikannya relatif kecil dibanding pertambahan durasi rata-rata (0,0465, 0,1152, dan 0,2494 s). Perbaikan terutama terlihat pada seed 2. Hasil ini belum cukup untuk menyimpulkan tren umum di luar dataset dan seed yang diuji.

3 Kesimpulan dan Saran

Tiga jenis algoritma wajib (Hill-Climbing, Simulated Annealing, dan Genetic Algorithm) berhasil dijalankan pada dataset 20 paket. Semua state akhir yang tercatat pada ringkasan eksperimen valid. Nilai terbaik yang teramati adalah 1240 pada GA dengan populasi 30, 50 generasi, dan seed 2. Hasil ini adalah terbaik di antara run yang diuji, bukan bukti bahwa optimum global telah ditemukan.

Pada eksperimen ini SA menghasilkan rentang nilai akhir yang kecil pada tingkat nilai tinggi (1215–1225), sedangkan GA pop 50/gen 30 memiliki rentang nol karena semua seed berakhir pada 1210. Steepest ascent memiliki durasi rata-rata terendah. GA pop 30/gen 50 menghasilkan nilai terbaik tunggal sekaligus rata-rata tertinggi di antara konfigurasi GA. Penambahan generasi dari 10 menjadi 50 memberi kenaikan rata-rata yang kecil dengan tambahan durasi. Kesimpulan tersebut terbatas pada satu dataset dan tiga seed, sehingga belum dapat digeneralisasi.

Keterbatasan dan pekerjaan lanjutan

Eksperimen belum membandingkan hasil dengan optimum global yang diketahui, belum menguji dataset lain, dan belum mengukur signifikansi statistik dari perbedaan antar-algoritma. Implementasi juga belum mencakup bonus fungsi objektif alternatif dan multi-truk.

Pengembangan berikutnya dapat menambah jumlah dataset dan seed, membandingkan hasil terhadap baseline atau solusi optimum yang dapat dibuktikan, serta menguji operator crossover yang lebih menjaga validitas state. Pengukuran durasi juga sebaiknya diulang pada lingkungan yang dicatat spesifikasinya.

4 Pembagian Tugas

Anggota	**Tugas**
Stevanus Agustaf Wongso (13524020)	Simulated Annealing, visualisasi state 3D, plot hasil, wiring visualisasi ke CLI, laporan
Jonathan Kris Wicaksono (13524023)	Representasi state, neighbor moves, Genetic Algorithm, wiring GA ke CLI, laporan
Philipp Hamara (13524101)	Evaluator dan initializer, data loader, Hill-Climbing, CLI utama, runner eksperimen, laporan

Tabel 6: Pembagian tugas tiap anggota kelompok.

Referensi

[1] Materi kuliah IF3170 Inteligensi Artifisial, Institut Teknologi Bandung, Semester I Tahun Ajaran 2026/2027.

[2] M. L. Khodra, "Beyond Classical Search: Classical vs Local Search," YouTube, 2021. [Daring]. Tersedia: https://www.youtube.com/watch?v=oGTa9kGE3P8. [Diakses 16 September 2026].

[3] M. L. Khodra, "Beyond Classical Search: State (Value, Neighbor)," YouTube, 2021. [Daring]. Tersedia: https://www.youtube.com/watch?v=_6rla37QorM. [Diakses 16 September 2026].

[4] M. L. Khodra, "Beyond Classical Search: Hill-climbing Search," YouTube, 2021. [Daring]. Tersedia: https://www.youtube.com/watch?v=L0b0CN3Fds8. [Diakses 16 September 2026].

[5] M. L. Khodra, "Beyond Classical Search: Simulated Annealing," YouTube, 2021. [Daring]. Tersedia: https://www.youtube.com/watch?v=AdhG7HnhYrk. [Diakses 16 September 2026].

[6] M. L. Khodra, "Beyond Classical Search: Genetic Algorithm," YouTube, 2021. [Daring]. Tersedia: https://www.youtube.com/watch?v=aI41TGo9J9c. [Diakses 16 September 2026].

Lampiran A: Form Penggunaan *Generative AI*

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, ~~saya~~/kami*:

Nama/NIM	:	Stevanus Agustaf Wongso / 13524020, Jonathan Kris Wicaksono / 13524023, Philipp Hamara / 13524101
Fakultas/Sekolah	:	STEI ITB
Kode Mata Kuliah	:	IF3170
Nama Mata Kuliah	:	Inteligensi Artifisial
Judul Tugas/Proposal	:	Tugas Besar 1 – Optimasi Pengisian Paket Truk Peti Kemas

Menyatakan bahwa ~~saya~~/kami* menggunakan/~~tidak menggunakan~~* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah: asisten AI untuk diskusi konsep pemrograman.

Lingkup Pekerjaan	**Digunakan?**	**Tingkat Penggunaan**
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, DeepL, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	–
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	–
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Kecil
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	–
Lainnya, Jelaskan:	–	–

~~Saya~~/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 20 Oktober 2026



(Stevanus Agustaf Wongso)
NIM 13524020



(Jonathan Kris Wicaksono)
NIM 13524023



(Philipp Hamara)
NIM 13524101